Chapitre 4

PROGRAMMATION LINÉAIRE

4.1 Introduction

Ce chapitre est consacré à la **programmation linéaire** qui permet de résoudre efficacement les problèmes d'optimisation où les contraintes et le critère s'expriment linéairement en fonction des variables (voir l'Exemple 1.2.1). Ce type de problème est extrêmement fréquent dans le domaine de la **recherche opérationnelle** (ou plus simplement RO). La RO est l'ensemble des méthodes scientifiques qui permettent de modéliser et analyser des situations complexes afin de prendre des décisions, sinon optimales, du moins les plus efficaces possibles. C'est un domaine au carrefour des mathématiques, de l'informatique et de l'ingénierie (au sens très large) où le mot "opérationnel" fait référence à la planification des opérations militaires car la RO est née pendant la seconde guerre mondiale. Depuis elle s'est très largement civilisée et elle est employée dans de très nombreuses applications pour l'industrie et les services, en économie et sciences de la gestion

La RO ne se limite pas du tout à la programmation linéaire et, bien au contraire, utilise des outils très divers, issus de plusieurs champs scientifiques : optimisation (continue et combinatoire), probabilités (algorithmes stochastiques), théorie des jeux, mathématiques discrètes, théorie des graphes, théorie de la complexité (informatique), et programmation par contraintes. Nous n'étudierons pas tous ces aspects de la RO, pas plus que nous ne toucherons aux questions de modélisation ou à la conception d'"heuristiques", lorsque des méthodes rigoureuses font défaut, ce qui représente souvent une part importante de la pratique en RO... Nous nous contentons ici de donner un éclairage très partiel sur les apports de l'optimisation, à travers la programmation linéaire, dans ce vaste domaine. Pour une introduction à la partie mathématisée de la RO nous renvoyons le lecteur vers l'ouvrage [5], issu d'un cours de troisième année à l'Ecole Polytechnique, ou bien vers [25].

La Section 4.2 est une présentation classique de la programmation linéaire. Au delà des (nombreux) cas où le modèle d'optimisation (fonction objectif et contraintes) est linéaire, rappelons que la programmation linéaire est aussi une brique de base dans de nombreux algorithmes d'optimisation qui linéarisent les problèmes à résoudre. La Section 4.3 est plus spécifique à la RO car elle étudie quelques exemples où l'on recherche des solutions à valeurs entières, et non pas seulement réelles, de

programmes linéaires. Lorsqu'on impose cette contrainte additionnelle (et très forte) de solutions entières, on parle alors de programmation linéaire en nombres entiers, voire d'optimisation combinatoire car il serait possible en théorie, mais illusoire en pratique, d'énumérer tous les candidats possibles pour trouver la solution optimale. Là aussi nous ne ferons qu'effleurer cette vaste question et nous renvoyons à [5] pour plus de détails.

4.2 Programmation linéaire

4.2.1 Définitions et propriétés

On veut résoudre le problème suivant, dit **programme linéaire sous forme standard**,

$$\inf_{x\in\mathbb{R}^n \text{tel que } Ax=b,\ x\geq 0} c\cdot x, \tag{4.1}$$

où A est une matrice de taille $m\times n$, $b\in\mathbb{R}^m$, $c\in\mathbb{R}^n$, et la contrainte $x\geq 0$ signifie que toutes les composantes de x sont positives ou nulles. Dans tout ce qui suit on supposera que $m\leq n$ et que le rang de A est exactement m. En effet, si $\mathrm{rg}(A)<m$, certaines lignes de A sont liées et deux possibilités se présentent : soit les contraintes (correspondantes à ces lignes) sont incompatibles, soit elles sont redondantes et on peut donc éliminer les lignes inutiles.

Le problème (4.1) semble être un cas particulier de programme linéaire puisque les contraintes d'inégalités sont seulement du type $x\geq 0$. Il n'en est rien, et tout programme linéaire du type

$$\inf_{x\in\mathbb{R}^n \text{tel que } Ax\geq b,\ A'x=b'} c\cdot x.$$

peut se mettre sous la forme standard (4.1) quitte à changer la taille des données. En effet, remarquons tout d'abord que les contraintes d'égalité $A'x=b'$ sont évidemment équivalentes aux contraintes d'inégalité $A'x\leq b'$ et $A'x\geq b'$. On peut donc se restreindre au cas suivant (qui ne contient que des contraintes d'inégalité)

$$\inf_{x\in\mathbb{R}^n \text{tel que } Ax\geq b} c\cdot x. \tag{4.2}$$

Dans (4.2) on peut remplacer la contrainte d'inégalité en introduisant de nouvelles variables, dites **d'écarts**, $\lambda\in\mathbb{R}^m$. La contrainte d'inégalité $Ax\geq b$ est alors équivalente à $Ax=b+\lambda$ avec $\lambda\geq 0$. Ainsi (4.2) est équivalent à

$$\inf_{(x,\lambda)\in\mathbb{R}^{(n+m)} \text{tel que } Ax=b+\lambda,\ \lambda\geq 0} c\cdot x. \tag{4.3}$$

Finalement, si on décompose chaque composante de x en partie positive et négative, c'est-à-dire si on pose $x=x^+-x^-$ avec $x^+=\max(0,x)$ et $x^-=-\min(0,x)$, on obtient que (4.2) est équivalent à

$$\inf_{(x^+,x^-,\lambda)\in\mathbb{R}^{(2n+m)} \text{tel que } Ax^+-Ax^-=b+\lambda,\ x^+\geq 0,x^-\geq 0,\lambda\geq 0} c\cdot(x^+-x^-). \tag{4.4}$$

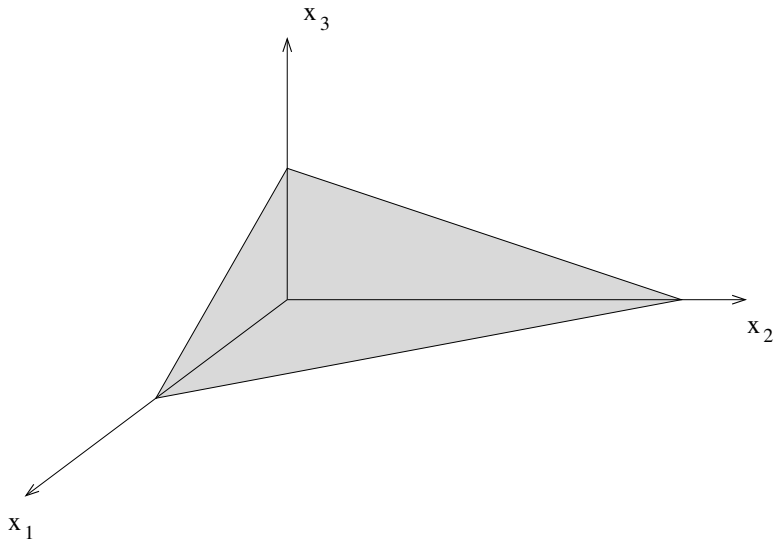


FIGURE 4.1 – Ensemble admissible pour l'exemple (4.5).

qui est bien sous forme standard (mais avec plus de variables). Il n'y a donc aucune perte de généralité à étudier le programme linéaire standard (4.1).

Nous avons déjà donné une motivation concrète de la programmation linéaire au début du Chapitre 1 (voir l'Exemple 1.2.1). Considérons pour l'instant un exemple simple qui va nous permettre de comprendre quelques aspects essentiels d'un programme linéaire

$$\min_{\substack{x_1\geq 0, x_2\geq 0, x_3\geq 0\\ 2x_1+x_2+3x_3=6}} x_1 + 4x_2 + 2x_3 \ . \tag{4.5}$$

Sur la Figure 4.1 nous avons tracé l'ensemble des (x_1, x_2, x_3) qui vérifient les contraintes : c'est un triangle plan T. C'est un fermé compact de $\mathbb{R}^3$, donc la fonction continue $x_1+4x_2+2x_3$ y atteint son minimum que l'on note M. Pour déterminer ce minimum on peut considérer la famille de plans parallèles $x_1 + 4x_2 + 2x_3 = c$ paramétrée par c. En augmentant la valeur de c à partir de $-\infty$, on "balaie" l'espace $\mathbb{R}^3$ jusqu'à atteindre le triangle T, et le minimum M est obtenu lorsque le plan "touche" ce triangle. Autrement dit, tout point de minimum de (4.5) est sur le bord du triangle T. Une autre façon de le voir est de dire que la fonction $x_1+4x_2+2x_3$ a un gradient non nul dans T donc ses extréma se trouvent sur le bord de T. Pour l'exemple (4.5) le point de minimum (unique) est le sommet $(0, 3, 0)$ de T. Nous verrons qu'il s'agit d'un fait général : un point de minimum (s'il existe) peut toujours se trouver en un des sommets de l'ensemble géométrique des vecteurs x qui vérifient les contraintes. Il "suffit" alors d'énumérer tous les sommets afin de trouver le minimum : c'est précisément ce que fait (de manière intelligente) l'algorithme du simplexe que nous verrons dans la prochaine sous-section.

Pour établir cette propriété en toute généralité pour le programme linéaire standard (4.1), nous avons besoin de quelques définitions qui permettent de préciser le

vocabulaire.

Définition 4.2.1 *L'ensemble X_{ad} des vecteurs de $\mathbb{R}^n$ qui satisfont les contraintes de (4.1), c'est-à-dire*

$$X_{ad} = \{x \in \mathbb{R}^n \text{ tel que } Ax = b,\ x \geq 0\},$$

est appelé ensemble des **solutions admissibles**. *On appelle sommet ou point extrémal de X_{ad} tout point $\overline{x} \in X_{ad}$ qui ne peut pas se décomposer en une combinaison convexe (non triviale) de deux autres points de X_{ad}, c'est-à-dire que, s'il existe $y, z \in X_{ad}$ et $\theta \in]0,1[$ tels que $\overline{x} = \theta y + (1-\theta)z$, alors $y = z = \overline{x}$.*

Remarque 4.2.2 Le vocabulaire de l'optimisation est trompeur pour les néophytes. On appelle solution (admissible) un vecteur qui satisfait les contraintes. Par contre, un vecteur qui atteint le minimum de (4.1) est appelé **solution optimale** (ou point de minimum). •

On vérifie facilement que l'ensemble X_{ad} est, d'une part convexe, d'autre part un **polyèdre** (éventuellement vide). Rappelons qu'un polyèdre est une intersection finie de demi-espaces de $\mathbb{R}^n$. Ses points extrémaux sont donc les sommets de ce polyèdre. Lorsque X_{ad} est vide, par convention on note que

$$\inf_{x \in \mathbb{R}^n \text{tel que } Ax=b,\ x \geq 0} c \cdot x = +\infty.$$

Lemme 4.2.3 *Il existe au moins une solution optimale (ou point de minimum) du programme linéaire standard (4.1) si et seulement si la valeur du minimum est finie*

$$-\infty < \inf_{x \in \mathbb{R}^n \text{tel que } Ax=b,\ x \geq 0} c \cdot x < +\infty.$$

Démonstration. Soit $(x^k)_{k \geq 1}$ une suite minimisante de (4.1). On introduit la matrice $\mathcal{A}$ définie par

$$\mathcal{A} = \begin{pmatrix} c^* \\ A \end{pmatrix}.$$

La suite $\mathcal{A}x^k$ appartient au cône suivant

$$C = \left\{ \sum_{i=1}^{n} x_i \mathcal{A}_i \text{ avec } x_i \geq 0 \right\},$$

où les $\mathcal{A}_i$ sont les colonnes de la matrice $\mathcal{A}$. D'après le Lemme de Farkas 2.5.19 le cône C est fermé, ce qui implique que

$$\lim_{k \to +\infty} \mathcal{A}x^k = \begin{pmatrix} z_0 \\ b \end{pmatrix} \in C,$$

donc il existe $\overline{x} \geq 0$ tel que

$$\begin{pmatrix} z_0 \\ b \end{pmatrix} = \begin{pmatrix} c \cdot \overline{x} \\ A\overline{x} \end{pmatrix},$$

et le minimum est atteint en $\overline{x}$. □

Définition 4.2.4 *On appelle* **base** *associée à (4.1) une base de $\mathbb{R}^m$ formée de m colonnes de A. On note B cette base qui est une sous-matrice de A, carrée d'ordre m inversible. Après permutation de ses colonnes on peut écrire A sous la forme (B, N) où N est une matrice de taille $m \times (n-m)$. De la même façon on peut décomposer x en (x_B, x_N) de sorte qu'on a*

$$Ax = Bx_B + Nx_N.$$

Les composantes du vecteur x_B sont appelées variables de base et celles de x_N variables hors base. Une **solution basique** *(ou de base) est un vecteur $x \in X_{ad}$ tel que $x_N = 0$. Si en plus l'une des composantes de x_B est nulle, on dit que la solution basique est* **dégénérée**.

La notion de solution basique correspond à celle de sommet de X_{ad}.

Lemme 4.2.5 *Les sommets du polyèdre X_{ad} sont exactement les solutions basiques.*

Démonstration. Si $x \in X_{ad}$ est une solution basique, dans une certaine base de $\mathbb{R}^n$ on a $x = (x_1, ..., x_m, 0, ..., 0)$, $A = (B, N)$ avec $B = (b_1, ..., b_m)$, une base de $\mathbb{R}^m$ telle que $\sum_{i=1}^m x_i b_i = b$. Supposons qu'il existe $0 < \theta < 1$ et $y, z \in X_{ad}$ tels que $x = \theta y + (1-\theta)z$. Nécessairement, les $n-m$ dernières composantes de y et z sont nulles et, comme y et z appartiennent à X_{ad}, on a $\sum_{i=1}^m y_i b_i = b$ et $\sum_{i=1}^m z_i b_i = b$. Par unicité de la décomposition dans une base, on en déduit que $x = y = z$, et donc x est un sommet de X_{ad}.

Réciproquement, si x est un sommet de X_{ad}, on note k le nombre de ses composantes non nulles, et après un éventuel réarrangement on a $b = \sum_{i=1}^k x_i a_i$ où les (a_i) sont les colonnes de A. Pour montrer que x est une solution basique il suffit de prouver que la famille $(a_1, ..., a_k)$ est libre dans $\mathbb{R}^m$ (on obtient une base B en complétant cette famille). Supposons que ce ne soit pas le cas : il existe alors $y \neq 0$ tel que $\sum_{i=1}^k y_i a_i = 0$ et $(y_{k+1}, ..., y_n) = 0$. Comme les composantes $(x_1, ..., x_k)$ sont strictement positives, il existe $\epsilon > 0$ (petit) tel que $(x + \epsilon y) \in X_{ad}$ et $(x - \epsilon y) \in X_{ad}$. Le fait que $x = (x + \epsilon y)/2 + (x - \epsilon y)/2$ contredit le caractère extrémal de x, donc x est une solution basique. □

Le résultat fondamental suivant nous dit qu'il est suffisant de chercher une solution optimale parmi les sommets du polyèdre X_{ad}.

Proposition 4.2.6 *S'il existe une solution optimale du programme linéaire standard (4.1), alors il existe une solution optimale basique.*

Démonstration. La démonstration est très similaire à celle du Lemme 4.2.5. Soit $x \in X_{ad}$ une solution optimale de (4.1). On note k le nombre de ses composantes non nulles, et après un éventuel réarrangement on a

$$b = \sum_{i=1}^k x_i a_i,$$

où les (a_i) sont les colonnes de A. Si la famille $(a_1, ..., a_k)$ est libre dans $\mathbb{R}^m$, alors x est une solution optimale basique. Si $(a_1, ..., a_k)$ est lié, alors il existe $y \neq 0$ tel que

$$\sum_{i=1}^{k} y_i a_i = 0 \text{ et } (y_{k+1}, ..., y_n) = 0.$$

Comme les composantes $(x_1, ..., x_k)$ sont strictement positives, il existe $\epsilon > 0$ tel que $(x \pm \epsilon y) \in X_{ad}$. Comme x est un point de minimum, on a nécessairement

$$c \cdot x \leq c \cdot (x \pm \epsilon y),$$

c'est-à-dire $c \cdot y = 0$. On définit alors une famille de points $z_\epsilon = x + \epsilon y$ paramétrée par ϵ. En partant de la valeur $\epsilon = 0$, si on augmente ou on diminue ϵ on reste dans l'ensemble X_{ad} jusqu'à une valeur ϵ_0 au delà de laquelle la contrainte $z_\epsilon \geq 0$ est violée. Autrement dit, $z_{\epsilon_0} \in X_{ad}$ possède au plus $(k-1)$ composantes non nulles et est encore solution optimale. On répète alors l'argument précédent avec $x = z_{\epsilon_0}$ et une famille de $(k-1)$ colonnes (a_i). A force de diminuer la taille de cette famille, on obtiendra finalement une famille libre et une solution optimale basique. □

Remarque 4.2.7 En appliquant la Proposition 4.2.6 lorsque $c = 0$ (toute solution admissible est alors optimale), on voit grâce au Lemme 4.2.5 que dès que X_{ad} est non-vide, X_{ad} a au moins un sommet. Cette propriété n'a pas lieu pour des polyèdres généraux (considérer un demi-plan de $\mathbb{R}^2$). •

Exercice 4.2.1 Résoudre le programme linéaire suivant

$$\max_{x_1 \geq 0, x_2 \geq 0} x_1 + 2x_2$$

sous les contraintes

$$\begin{cases} -3x_1 + 2x_2 & \leq & 2, \\ -x_1 + 2x_2 & \leq & 4, \\ x_1 + x_2 & \leq & 5. \end{cases}$$

En pratique le nombre de sommets du polyèdre X_{ad} est gigantesque car il peut être exponentiel par rapport au nombre de variables. On le vérifie sur un exemple dans l'exercice suivant.

Exercice 4.2.2 Montrer que l'on peut choisir la matrice A de taille $m \times n$ et le vecteur $b \in \mathbb{R}^m$ de telle façon que X_{ad} soit le cube unité $[0,1]^{n-m}$ dans le sous-espace affine de dimension $n - m$ défini par $Ax = b$. En déduire que le nombre de sommets de X_{ad} est alors 2^{n-m}.

4.2.2 Algorithme du simplexe

L'algorithme du simplexe est dû à G. Dantzig dans les années 1940. Il consiste à parcourir les sommets du polyèdre des solutions admissibles jusqu'à ce qu'on trouve

une solution optimale (ce qui est garanti si le programme linéaire admet effectivement une solution optimale). L'algorithme du simplexe ne se contente pas d'énumérer tous les sommets, il décroît la valeur de la fonction $c \cdot x$ en passant d'un sommet au suivant.

On considère le programme linéaire standard (4.1). Rappelons qu'un sommet (ou solution basique) de l'ensemble des solutions admissibles X_{ad} est caractérisé par une base B (m colonnes libres de A). Après permutation de ses colonnes, on peut écrire

$$A = (B, N) \text{ et } x = (x_B, x_N),$$

de sorte qu'on a $Ax = Bx_B + Nx_N$. Toute solution admissible peut s'écrire $x_B = B^{-1}(b - Nx_N) \geq 0$ et $x_N \geq 0$. Le sommet associé à B est défini (s'il existe) par $\overline{x}_N = 0$ et $\overline{x}_B = B^{-1}b \geq 0$. Si on décompose aussi $c = (c_B, c_N)$ dans cette base, alors on peut comparer le coût d'une solution admissible quelconque x avec celui de la solution basique $\overline{x}$

$$c \cdot x - c \cdot \overline{x} = c_B \cdot B^{-1}(b - Nx_N) + c_N \cdot x_N - c_B \cdot B^{-1}b = (c_N - N^*(B^{-1})^* c_B) \cdot x_N. \quad (4.6)$$

On en déduit la condition d'optimalité suivante.

Proposition 4.2.8 *Supposons que la solution basique associée à B est non dégénérée, c'est-à-dire que $B^{-1}b > 0$. Une condition nécessaire et suffisante pour que cette solution basique associée à B soit optimale est que*

$$\tilde{c}_N = c_N - N^*(B^{-1})^* c_B \geq 0. \quad (4.7)$$

Le vecteur $\tilde{c}_N$ est appelé **vecteur des coûts réduits**.

Démonstration. Soit $\overline{x}$ une solution basique non dégénérée associée à B. Si $\tilde{c}_N \geq 0$, alors pour toute solution admissible x (4.6) implique que

$$c \cdot x - c \cdot \overline{x} = \tilde{c}_N \cdot x_N \geq 0 \,,$$

puisque $x_N \geq 0$. Donc la condition (4.7) est suffisante pour que $\overline{x}$ soit optimal. Réciproquement, supposons qu'il existe une composante i de $\tilde{c}_N$ qui soit strictement négative, $(\tilde{c}_N \cdot e_i) < 0$. Pour $\epsilon > 0$ on définit alors un vecteur $x(\epsilon)$ par $x_N(\epsilon) = \epsilon e_i$ et $x_B(\epsilon) = B^{-1}(b - Nx_N(\epsilon))$. Par construction $Ax(\epsilon) = b$ et, comme $B^{-1}b > 0$, pour des valeurs suffisamment petites de ϵ on a $x(\epsilon) \geq 0$, donc $x(\epsilon) \in X_{ad}$. D'autre part, $x(0) = \overline{x}$ et, comme $\epsilon > 0$, on a

$$c \cdot x(\epsilon) = c \cdot x(0) + \epsilon(\tilde{c}_N \cdot e_i) < c \cdot \overline{x},$$

ce qui montre que $\overline{x}$ n'est pas optimal. Donc la condition (4.7) est nécessaire. □

Remarque 4.2.9 Dans le cadre de la Proposition 4.2.8, si la solution basique considérée est dégénérée, la condition (4.7) reste suffisante mais n'est plus nécessaire. •

On déduit de la Proposition 4.2.8 une méthode pratique pour décroître la valeur de la fonction coût $c \cdot x$ à partir d'une solution basique $\overline{x}$ (non dégénérée et non optimale). Comme $\overline{x}$ est non-optimale, il existe une composante du vecteur des coûts réduits $\tilde{c}_N$ telle que $\tilde{c}_N \cdot e_i < 0$. On définit alors $x(\epsilon)$ comme ci-dessus. Puisque le coût décroît linéairement avec ϵ, on a intérêt à prendre la plus grande valeur possible de ϵ telle que l'on reste dans X_{ad}. C'est le principe de l'algorithme du simplexe que nous présentons maintenant.

Algorithme du simplexe

— Initialisation (phase I) : on cherche une base initiale B^0 telle que la solution basique associée x^0 soit admissible

$$x^0 = \begin{pmatrix} (B^0)^{-1}b \\ 0 \end{pmatrix} \geq 0.$$

— Itérations (phase II) : à l'étape $k \geq 0$, on dispose d'une base B^k et d'une solution basique admissible x^k. On calcule le coût réduit $\tilde{c}_N^k = c_N^k - (N^k)^*(B^{k^{-1}})^* c_B^k$. Si $\tilde{c}_N^k \geq 0$, alors x^k est optimal et l'algorithme est fini. Sinon, il existe une variable hors-base d'indice i telle que $(\tilde{c}_N \cdot e_i) < 0$, et on note a_i la colonne correspondante de A. On pose

$$x^k(\epsilon) = (x_B^k(\epsilon), x_N^k(\epsilon)) \text{ avec } x_N^k(\epsilon) = \epsilon e_i \ , \ x_B^k(\epsilon) = (B^k)^{-1}(b - \epsilon a_i).$$

  — Soit on peut choisir $\epsilon > 0$ aussi grand que l'on veut avec $x^k(\epsilon) \in X_{ad}$. Dans ce cas, le minimum du programme linéaire est $-\infty$.
  — Soit il existe une valeur maximale $\epsilon^k \geq 0$ et un indice j tels que la j-ème composante de $x^k(\epsilon^k)$ s'annule. On obtient ainsi une nouvelle solution admissible basique

$$x^{k+1} = x^k(\epsilon^k),$$

  correspondant à une nouvelle base B^{k+1} déduite de B^k en remplaçant sa j-ème colonne par la colonne a_i. La solution admissible x^{k+1} a un coût inférieur ou égal à celui de x^k.

Il reste un certain nombres de points pratiques à préciser dans l'algorithme du simplexe. Nous les passons rapidement en revue.

Dégénérescence et cyclage

On a toujours $c \cdot x^{k+1} \leq c \cdot x^k$, mais il peut y avoir égalité si la solution admissible basique x^k est dégénérée, auquel cas on trouve que $\epsilon^k = 0$ (si x^k n'est pas dégénérée, la démonstration de la Proposition 4.2.8 garantit une inégalité stricte). On a donc changé de base sans améliorer le coût : c'est le phénomène du cyclage qui peut empêcher l'algorithme de converger. Il existe des moyens de s'en prémunir, mais en pratique le cyclage n'apparaît jamais.

En l'absence de cyclage, l'algorithme du simplexe parcourt un sous-ensemble des sommets de X_{ad} en diminuant de façon stricte le coût. Comme il y a un nombre

fini de sommets, l'algorithme doit nécessairement trouver un sommet optimal de coût minimal. On a donc démontré le résultat suivant.

Lemme 4.2.10 *Si toutes solutions admissibles basiques x^k produites par l'algorithme du simplexe sont non dégénérées, alors l'algorithme converge en un nombre fini d'étapes.*

A priori le nombre d'itérations de l'algorithme du simplexe peut être aussi grand que le nombre de sommets (qui est exponentiel par rapport au nombre de variables n ; voir l'Exercice 4.2.2). Bien qu'il existe des exemples (académiques) où c'est effectivement le cas, en pratique cet algorithme converge en un nombre d'étapes qui est une fonction polynomiale de n.

Choix du changement de base

S'il y a plusieurs composantes du vecteur coût réduit $\tilde{c}_N^k$ strictement négatives, il faut faire un choix dans l'algorithme. Plusieurs stratégies sont possibles, mais en général on choisit la plus négative.

Initialisation

Comment trouver une solution admissible basique lors de l'initialisation ? (Rappelons que la condition d'admissibilité $x_B = B^{-1}b \geq 0$ n'est pas évidente en général.) Soit on en connaît une à cause de la structure du problème. Par exemple, pour le problème (4.4) qui possède m variables d'écart, $-\mathrm{Id}_m$ est une base de la matrice "globale" des contraintes d'égalité de (4.4). Si de plus $b \leq 0$, le vecteur $(x_+^0, x_-^0, \lambda^0) = (0, 0, -b)$ est alors une solution admissible basique pour (4.4).

Dans le cas général, on introduit une nouvelle variable $y \in \mathbb{R}^m$, un nouveau vecteur coût $k = (1, ..., 1)$ et un nouveau programme linéaire

$$\inf_{\substack{x \geq 0,\ y \geq 0 \\ Ax + y = b}} k \cdot y \,, \tag{4.8}$$

où on a préalablement multiplié par -1 toutes les contraintes d'égalité correspondant à des composantes négatives de b de telles sortes que $b \geq 0$. Le vecteur $(x^0, y^0) = (0, b)$ est une solution admissible basique pour ce problème. S'il existe une solution admissible du programme linéaire original (4.1), alors il existe au moins une solution optimale de (4.8) et toutes les solutions optimales (x, y) vérifient nécessairement $y = 0$ et x est solution admissible de (4.1). En appliquant l'algorithme du simplexe à (4.8), on trouve ainsi une solution admissible basique pour (4.1) s'il en existe une. S'il n'en existe pas (c'est-à-dire si $X_{ad} = \emptyset$), on le détecte car le minimum de (4.8) est atteint par un vecteur (x, y) avec $y \neq 0$.

Inversion de la base

Tel que nous l'avons décrit l'algorithme du simplexe demande l'inversion à chaque étape de la base B^k, ce qui peut être très coûteux pour les problèmes de

grande taille (avec beaucoup de contraintes puisque l'ordre de B^k est égal au nombre de contraintes). On peut tirer parti du fait que B^{k+1} ne diffère de B^k que par une colonne pour mettre au point une meilleur stratégie. En effet, si c'est la j-ème colonne qui change, on a

$$B^{k+1} = B^k E^k \quad \text{avec } E^k = \begin{pmatrix} 1 & & & l_1 & & \\ & \ddots & & \vdots & & 0 \\ & & 1 & \vdots & & \\ & & & l_j & & \\ & & & \vdots & 1 & \\ & 0 & & \vdots & & \ddots \\ & & & l_n & & 1 \end{pmatrix},$$

et E^k est facile à inverser

$$(E^k)^{-1} = \frac{1}{l_j} \begin{pmatrix} 1 & & & -l_1 & & \\ & \ddots & & \vdots & & 0 \\ & & 1 & -l_{j-1} & & \\ & & & 1 & & \\ & & & -l_{j+1} & 1 & \\ & 0 & & \vdots & & \ddots \\ & & & -l_n & & 1 \end{pmatrix}.$$

On utilise donc la formule, sous forme factorisée,

$$(B^k)^{-1} = (E^{k-1})^{-1}(E^{k-2})^{-1} \cdots (E^0)^{-1}(B^0)^{-1}.$$

Exercice 4.2.3 Résoudre par l'algorithme du simplexe le programme linéaire

$$\min_{x_1 \geq 0,\ x_2 \geq 0,\ x_3 \geq 0,\ x_4 \geq 0,\ x_5 \geq 0} x_1 + 2x_2$$

sous les contraintes

$$\begin{cases} -3x_1 + 2x_2 + x_3 = 2, \\ -x_1 + 2x_2 + x_4 = 4, \\ x_1 + x_2 + x_5 = 5. \end{cases}$$

Exercice 4.2.4 Résoudre par l'algorithme du simplexe le programme linéaire

$$\min_{x_1 \geq 0,\ x_2 \geq 0} 2x_1 - x_2$$

sous les contraintes $x_1 + x_2 \leq 1$ et $x_2 - x_1 \leq 1/2$ (on pourra s'aider d'un dessin et introduire des variables d'écart).

Exercice 4.2.5 Résoudre par l'algorithme du simplexe le programme linéaire

$$\min_{x_1 \geq 0,\ x_2 \geq 0,\ x_3 \geq 0,\ x_4 \geq 0} 3x_3 - x_4$$

sous les contraintes

$$\begin{cases} x_1 - 3x_3 + 3x_4 = 6, \\ x_2 - 8x_3 + 4x_4 = 4. \end{cases}$$

4.2.3 Algorithmes de points intérieurs

Depuis les travaux de Khachian et Karmarkar au début des années 1980, une nouvelle classe d'algorithmes, dits de points intérieurs, est apparu pour résoudre des programmes linéaires. Le nom de cette classe d'algorithmes vient de ce qu'au contraire de la méthode du simplexe (qui, parcourant les sommets, reste sur le bord du polyèdre X_{ad}) ces algorithmes de points intérieurs évoluent à l'intérieur de X_{ad} et ne rejoignent son bord qu'à convergence. Nous allons décrire ici un de ces algorithmes que l'on appelle aussi **algorithme de trajectoire centrale**. Il y a deux idées nouvelles dans cette méthode : premièrement, on pénalise certaines contraintes à l'aide de potentiels ou fonctions "barrières" ; deuxièmement, on utilise une méthode de Newton pour passer d'une itérée à la suivante.

Décrivons cette méthode sur le programme linéaire standard

$$\inf_{x\in\mathbb{R}^n \text{tel que } Ax=b,\ x\geq 0} c\cdot x. \tag{4.9}$$

On définit un potentiel logarithmique pour $x > 0$

$$\pi(x) = -\sum_{i=1}^{n} \log x_i. \tag{4.10}$$

Pour un paramètre de pénalisation $\mu > 0$, on introduit le problème strictement convexe

$$\min_{x\in\mathbb{R}^n \text{tel que } Ax=b,\ x>0} \mu\pi(x) + c\cdot x. \tag{4.11}$$

Remarquons qu'en pratique la contrainte $x > 0$ n'en est pas une car elle n'est jamais active : quand on minimise (4.11) on ne peut pas "s'approcher" du bord de $x > 0$ sous peine de faire "exploser" le potentiel $\pi(x)$ vers $+\infty$. Il s'agit donc d'un exemple de méthode de **pénalisation intérieure**.

Le principe de l'algorithme de trajectoire centrale est de minimiser (4.11) par une méthode de Newton pour des valeurs de plus en plus petites de μ. En effet, lorsque μ tend vers zéro, le problème pénalisé (4.11) tend vers le programme linéaire (4.9).

Lemme 4.2.11 *On définit l'ensemble admissible de (4.11) par*

$$X_{ad}^0 = \{x \in \mathbb{R}^n \text{ tel que } Ax = b,\ x > 0\}.$$

On suppose que X_{ad}^0 est borné non vide. Alors le problème pénalisé (4.11) admet une unique solution optimale x^μ. De plus, si (4.9) admet une unique solution optimale x^0, alors x^μ converge vers x^0 lorsque μ tend vers zéro.

Remarque 4.2.12 L'hypothèse que X_{ad}^0 est non vide est nécessaire pour que (4.11) ait au moins une solution admissible. Il est facile de construire des exemples d'ensemble admissible X_{ad} non vide mais tel que X_{ad}^0 est vide (voir l'Exercice 4.2.6). L'hypothèse que X_{ad}^0 est borné est équivalente à celle que $X_{ad} = \overline{X_{ad}^0}$ est borné.

Cette hypothèse est nécessaire car on peut construire des exemples où, X_{ad} n'étant pas borné, le programme linéaire (4.9) admet une solution optimale x^0 mais le problème pénalisé (4.11) n'admet pas de solution optimale (voir l'Exercice 4.2.6 pour un tel contre-exemple). •

Démonstration. La fonction $\mu\pi(x) + c \cdot x$ est strictement convexe sur $\mathbb{R}^n_+$ donc, s'il existe un point de minimum à (4.11), il est unique. Par contre, cette fonction n'est pas infinie à l'infini donc on a besoin de l'hypothèse que X^0_{ad} est borné. Par ailleurs, X^0_{ad} est convexe mais pas fermé, donc on ne peut pas utiliser directement le Théorème 2.2.1 bien que J soit continue et X^0_{ad} borné. Soit $x^n \in X^0_{ad}$ une suite minimisante de (4.11). Comme $\overline{X^0_{ad}} = X_{ad}$ est borné, on peut en extraire une sous-suite, toujours notée x^n, qui converge vers une limite x^μ dans X_{ad}. En fait, toutes les composantes de x^μ sont strictement positives et $x^\mu \in X^0_{ad}$ car sinon

$$\lim_{n\to+\infty} \pi(x^n) = +\infty,$$

ce qui serait une contradiction avec le fait que la valeur minimale de (4.11) est bornée supérieurement puisque X^0_{ad} n'est pas vide. Comme la fonction $\mu\pi(x) + c \cdot x$ est continue, on a donc obtenu

$$\mu\pi(x^\mu) + c \cdot x^\mu = \lim_{n\to+\infty} \mu\pi(x^n) + c \cdot x^n = \min_{x\in X^0_{ad}} \mu\pi(x) + c \cdot x.$$

Autrement dit, x^μ est la solution optimale de (4.11). On écrit les conditions d'optimalité (comme toujours dans ce chapitre, on suppose que la matrice A, de taille $m \times n$, est de rang $m \leq n$, donc les contraintes sont qualifiées) pour x^μ : il existe un multiplicateur de Lagrange $p^\mu \in \mathbb{R}^m$ tel que

$$Ax^\mu = b, \quad c - \mu\frac{1}{x^\mu} + A^* p^\mu = 0, \tag{4.12}$$

où $1/x$ désigne le vecteur de $\mathbb{R}^n$ de composantes $1/x_i$. On veut passer à la limite, quand $\mu \to 0$, dans (4.12). Comme l'ensemble X_{ad} est borné, la suite x^μ est bornée et on peut en extraire une sous-suite qui converge vers une limite $x^0 \in X_{ad}$ car X_{ad} est fermé. Par contre, en général la suite p^μ n'est pas bornée, quand $\mu \to 0$, et nous allons simplement montrer que la suite A^*p^μ est bornée dans $\mathbb{R}^n$. On déduit de (4.12) que

$$A^* p^\mu + c \geq 0. \tag{4.13}$$

Au vu de (4.13) les composantes de A^*p^μ sont bornées inférieurement. Montrons qu'elles sont aussi bornées supérieurement. Soit $y \in X^0_{ad}$ qui est supposé non vide. On multiplie la contrainte $Ay = b$ par p^μ pour obtenir

$$y \cdot A^* p^\mu = b \cdot p^\mu.$$

Par ailleurs, si on multiplie la deuxième égalité de (4.12) par x^μ on obtient

$$c \cdot x^\mu - n\mu + A^* p^\mu \cdot x^\mu = c \cdot x^\mu - n\mu + p^\mu \cdot b = 0, \tag{4.14}$$

donc

$$y \cdot A^* p^\mu = n\mu - c \cdot x^\mu \leq C,$$

car la suite x^μ est bornée. Comme $y > 0$, les composantes de $A^* p^\mu$ sont bornées supérieurement. Autrement dit, $A^* p^\mu$ est une suite bornée dans $\mathbb{R}^n$. On peut donc en extraire une sous-suite qui converge. Par ailleurs, elle appartient à $\mathrm{Im} A^*$ qui est un sous-espace vectoriel fermé, donc il existe $p^0 \in \mathbb{R}^m$ tel que la sous-suite $A^* p^\mu$ converge vers $A^* p^0$.

On passe alors à la limite dans la première égalité de (4.12), dans (4.13) et (4.14)

$$Ax^0 = b, \quad A^* p^0 + c \geq 0, \quad c \cdot x^0 + A^* p^0 \cdot x^0 = 0.$$

Il s'agit des conditions d'optimalité de (4.9), donc en vertu du théorème de Kuhn et Tucker (voir le Théorème 4.2.13 plus bas) x^0 est la solution optimale de (4.9) et p^0 est un multiplicateur de Lagrange pour ce problème. Si on suppose que (4.9) admet une solution unique, alors toute la suite x^μ converge vers x^0 lorsque μ tend vers zéro. □

Exercice 4.2.6 Donner un exemple de programme linéaire (4.9) qui admet une solution optimale x^0 mais pour lequel l'ensemble admissible X_{ad} n'est pas borné et le problème pénalisé (4.11) n'admet pas de solution optimale. Donner un autre exemple d'ensemble admissible X_{ad} non vide mais tel que X_{ad}^0 est vide.

4.2.4 Dualité

La théorie de la dualité (déjà évoquée lors de la Sous-section 2.6.3) est très utile en programmation linéaire. Considérons à nouveau le programme linéaire standard que nous appellerons primal (par opposition au dual)

$$\inf_{x \in \mathbb{R}^n \text{tel que } Ax=b,\ x \geq 0} c \cdot x, \tag{4.15}$$

où A est une matrice de taille $m \times n$, $b \in \mathbb{R}^m$, et $c \in \mathbb{R}^n$. Pour $p \in \mathbb{R}^m$, on introduit le Lagrangien de (4.15)

$$L(x, p) = c \cdot x + p \cdot (b - Ax), \tag{4.16}$$

où l'on a seulement "dualisé" les contraintes d'égalité. On introduit la fonction duale associée

$$G(p) = \min_{x \geq 0} L(x, p),$$

qui, après calcul, vaut

$$G(p) = \begin{cases} p \cdot b & \text{si } A^* p - c \leq 0 \\ -\infty & \text{sinon.} \end{cases} \tag{4.17}$$

Le problème dual de (4.15) est donc

$$\sup_{p \in \mathbb{R}^m \text{tel que } A^* p - c \leq 0} p \cdot b. \tag{4.18}$$

L'espace de solutions admissibles du problème dual (4.18) est noté

$$P_{ad} = \{p \in \mathbb{R}^m \text{ tel que } A^*p - c \leq 0\}.$$

Rappelons que l'espace de solutions admissibles de (4.15) est

$$X_{ad} = \{x \in \mathbb{R}^n \text{ tel que } Ax = b,\ x \geq 0\}.$$

Les programmes linéaires (4.15) et (4.18) sont dits en **dualité**. L'intérêt de cette notion vient du résultat suivant qui est un cas particulier du Théorème de dualité 2.6.12.

Théorème 4.2.13 *Si (4.15) ou (4.18) a une valeur optimale finie, alors il existe $\overline{x} \in X_{ad}$ solution optimale de (4.15) et $\overline{p} \in P_{ad}$ solution optimale de (4.18) qui vérifient*

$$\left(\min_{x\in\mathbb{R}^n \text{tel que } Ax=b,\ x\geq 0} c\cdot x\right) = c\cdot\overline{x} = \overline{p}\cdot b = \left(\max_{p\in\mathbb{R}^m \text{tel que } A^*p-c\leq 0} p\cdot b\right) \tag{4.19}$$

De plus, $\overline{x}$ et $\overline{p}$ sont solutions optimales de (4.15) et (4.18) si et seulement si elles vérifient les conditions d'optimalité de Kuhn et Tucker

$$A\overline{x} = b,\ \overline{x} \geq 0,\ A^*\overline{p} - c \leq 0,\ \overline{x}\cdot(c - A^*\overline{p}) = 0. \tag{4.20}$$

Si (4.15) ou (4.18) a une valeur optimale infinie, alors l'ensemble des solutions admissibles de l'autre problème est vide.

Remarque 4.2.14 Une conséquence immédiate du Théorème 4.2.13 de dualité est que, si $x \in X_{ad}$ et $p \in P_{ad}$ sont deux solutions admissibles de (4.15) et (4.18), respectivement, elles vérifient

$$c\cdot x \geq b\cdot p.$$

De même, si $\overline{x} \in X_{ad}$ et $\overline{p} \in P_{ad}$ vérifient

$$c\cdot\overline{x} = b\cdot\overline{p}$$

alors $\overline{x}$ est solution optimale de (4.15) et $\overline{p}$ de (4.18). Ces deux propriétés permettent de trouver facilement des bornes pour les valeurs optimales de (4.15) et (4.18), et de tester si un couple $(\overline{x}, \overline{p})$ est optimal. •

Démonstration. Supposons que X_{ad} et P_{ad} sont non vides. Soit $x \in X_{ad}$ et $p \in P_{ad}$. Comme $x \geq 0$ et $A^*p \leq c$, on a

$$c\cdot x \geq A^*p\cdot x = p\cdot Ax = p\cdot b,$$

puisque $Ax = b$. En particulier, cette inégalité implique que les valeurs optimales des deux problèmes, primal et dual, sont finies, donc qu'ils admettent des solutions optimales en vertu du Lemme 4.2.3. L'égalité (4.19) et la condition d'optimalité (4.20) sont alors une conséquence du Théorème de dualité 2.6.12.

Supposons maintenant que l'un des deux problèmes primal ou dual admet une valeur optimale finie. Pour fixer les idées, admettons qu'il s'agisse du problème dual (un argument symétrique fonctionne pour le problème primal). Alors, le Lemme 4.2.3 affirme qu'il existe une solution optimale $\overline{p}$ de (4.18). Si X_{ad} n'est pas vide, on se retrouve dans la situation précédente ce qui finit la démonstration. Montrons donc que X_{ad} n'est pas vide en utilisant encore le Lemme de Farkas 2.5.19. Pour $p \in \mathbb{R}^m$, on introduit les vecteurs de $\mathbb{R}^{m+1}$

$$\tilde{b} = \begin{pmatrix} b \\ -b \cdot \overline{p} \end{pmatrix} \quad \text{et} \quad \tilde{p} = \begin{pmatrix} p \\ 1 \end{pmatrix}.$$

On vérifie que $\tilde{b} \cdot \tilde{p} = b \cdot p - b \cdot \overline{p} \leq 0$, pour tout $p \in P_{ad}$. D'autre part, la condition $p \in P_{ad}$ peut se réécrire

$$\tilde{p} \in C = \left\{ \tilde{p} \in \mathbb{R}^{m+1} \text{ tel que } \tilde{p}_{m+1} = 1,\ \tilde{A}^* \tilde{p} \leq 0 \right\} \text{ avec } \tilde{A} = \begin{pmatrix} A \\ -c^* \end{pmatrix}.$$

Comme $\tilde{b} \cdot \tilde{p} \leq 0$ pour tout $\tilde{p} \in C$, le Lemme de Farkas 2.5.19 nous dit qu'il existe $\tilde{x} \in \mathbb{R}^n$ tel que $\tilde{x} \geq 0$ et $\tilde{b} = \tilde{A}\tilde{x}$, c'est-à-dire que $\tilde{x} \in X_{ad}$ qui n'est donc pas vide.

Finalement, supposons que la valeur optimale du problème primal est (4.15) $-\infty$. Si P_{ad} n'est pas vide, pour tout $x \in X_{ad}$ et tout $p \in P_{ad}$, on a $c \cdot x \geq b \cdot p$. En prenant une suite minimisante dans X_{ad} on obtient $b \cdot p = -\infty$, ce qui absurde. Donc P_{ad} est vide. Un raisonnement similaire montre que, si la valeur optimale de (4.15) est infinie, alors X_{ad} est vide. □

L'intérêt de la dualité pour résoudre le programme linéaire (4.15) est multiple. D'une part, selon l'algorithme choisi, il peut être plus facile de résoudre le problème dual (4.18) (qui a m variables et n contraintes d'inégalités) que le problème primal (4.15) (qui a n variables, m contraintes d'égalités et n contraintes d'inégalités). D'autre part, on peut construire des algorithmes numériques très efficaces pour la résolution de (4.15) qui utilisent les deux formes primale et duale du programme linéaire.

Exercice 4.2.7 Utiliser la dualité pour résoudre "à la main" (et sans calculs !) le programme linéaire

$$\min_{x_1 \geq 0,\ x_2 \geq 0,\ x_3 \geq 0,\ x_4 \geq 0} 8x_1 + 9x_2 + 4x_3 + 6x_4$$

sous les contraintes

$$\begin{cases} 4x_1 + x_2 + x_3 + 2x_4 \geq 1 \\ x_1 + 3x_2 + 2x_3 + x_4 \geq 1 \end{cases}$$

Exercice 4.2.8 Trouver le problème dual de (4.15) lorsqu'on dualise aussi la contrainte $x \geq 0$, c'est-à-dire qu'on introduit le Lagrangien

$$L(x, p, q) = c \cdot x + p \cdot (b - Ax) - q \cdot x$$

avec $q \in \mathbb{R}^n$ tel que $q \geq 0$. Comparer avec (4.18) et interpréter la nouvelle variable duale q. En déduire qu'il n'y a pas d'intérêt à "dualiser" aussi la contrainte $x \geq 0$.

Exercice 4.2.9 Vérifier que le problème dual de (4.18) est à nouveau (4.15).

Exercice 4.2.10 Soit $v \in \mathbb{R}^n$, $c \in \mathbb{R}^n$, A une matrice $m \times n$ et $b \in \mathbb{R}^m$. On considère le programme linéaire

$$\inf_{\substack{v \geq 0 \\ Av \leq b}} c \cdot v \,. \tag{4.21}$$

Montrer que le problème dual peut se mettre sous la forme suivante, avec $q \in \mathbb{R}^m$

$$\sup_{\substack{q \geq 0 \\ A^* q + c \geq 0}} -b \cdot q \,. \tag{4.22}$$

Soient v et q des solutions admissibles de (4.21) et (4.22), respectivement. Montrer que v et q sont des solutions optimales si, et seulement si,

$$(c + A^* q) \cdot v = 0 \quad \text{et} \quad (b - Av) \cdot q = 0. \tag{4.23}$$

Les deux égalités de (4.23) sont appelées **conditions des écarts complémentaires** (primales et duales, respectivement).

4.3 Vers la programmation linéaire en nombres entiers

Jusqu'ici nous avons considéré des problèmes d'**optimisation continue**, c'est-à-dire que la variable d'optimisation variait continûment dans un ensemble admissible inclus dans $\mathbb{R}^n$. Dans le cas de la programmation linéaire, cet ensemble admissible X_{ad} est un polyèdre. La programmation linéaire **en nombres entiers** consiste à abandonner ce point de vue continu et à rajouter la contrainte que la solution doit appartenir à $\mathbb{Z}^n$. Dans le cas d'un programme linéaire sous forme standard (4.1) on considère donc

$$\inf_{x \in \mathbb{Z}^n \text{ tel que } Ax = b,\ x \geq 0} c \cdot x, \tag{4.24}$$

où A est une matrice de taille $m \times n$ (avec $m \leq n$ et $\mathrm{rg}(A) = m$), $b \in \mathbb{R}^m$ et $c \in \mathbb{R}^n$. On dit que (4.24) est un programme linéaire en nombres entiers, ou bien en variables discrètes, ou encore un problème combinatoire puisqu'on peut, en théorie, énumérer toutes les solutions possibles afin de trouver la solution optimale. Evidemment, le nombre de solutions entières possibles est exponentiel dans la taille n, m des données et il n'est en général pas possible de faire cette énumération en pratique.

En toute généralité le problème (4.24) est extrêmement difficile à résoudre, en particulier puisque, l'espace des solutions admissibles étant discret, on ne peut pas utiliser la notion de gradient pour tester l'optimalité d'une solution ou pour passer d'une solution à une autre meilleure. Nous ne disons rien des méthodes générales pour résoudre (4.24) et nous nous contentons de traiter deux cas particuliers, importants du point de vue des applications, pour lesquels la programmation linéaire continue, vue à la section précédente, est utile et efficace. Ces deux cas particuliers correspondent à un petit miracle que nous expliquons tout de suite. En général,

puisque $\mathbb{Z}^n$ est un sous-ensemble (très petit !) de $\mathbb{R}^n$, la valeur minimale de (4.24) est plus grande que la valeur minimale de (4.1), qui est le même problème où $\mathbb{Z}^n$ est remplacé par $\mathbb{R}^n$ (parfois on appelle (4.1) problème relaxé, ou relâché, de (4.24) car on abandonne la contrainte que x doit être à valeurs entières). Le miracle est que, justement pour ces deux cas, la valeur minimale est la même ! Plus précisément, on verra qu'en utilisant l'algorithme du simplexe pour (4.1) on obtient une solution de (4.24), qui est donc à valeurs entières. Bien entendu, cela n'est pas vrai sans des hypothèses de structure très fortes sur les données. Ces deux cas, qui peuvent sembler exceptionnels, apparaissent en fait naturellement dans un certain nombre de problèmes combinatoires concrets : affectations, plus courts chemins et plus généralement problèmes de flots à coût minimum.

4.3.1 Matrices totalement unimodulaires

Commençons par étudier à quelles conditions sur les données entières A et b les solutions admissibles $x \in X_{ad}$ ou plus généralement les solutions du système linéaire $Ax = b$ sont, elles aussi entières. Etudions d'abord le cas des matrices carrées dans le lemme suivant.

Lemme 4.3.1 *Pour une matrice inversible $A \in \mathbb{Z}^{n\times n}$ les deux propriétés suivantes sont équivalentes :*

1. $\det A = \pm 1$ *;*
2. *pour tout $b \in \mathbb{Z}^n$, on a $A^{-1}b \in \mathbb{Z}^n$.*

Démonstration. Au vu des formules de Cramer pour la résolution du système linéaire $Ax = b$, il est clair que si $\det A = \pm 1$, alors $A^{-1}b \in \mathbb{Z}^n$ pour tout second membre $b \in \mathbb{Z}^n$. Pour prouver la réciproque montrons tout d'abord que $A^{-1} \in \mathbb{Z}^{n\times n}$. En effet, si on choisit b comme le i-ème vecteur de la base canonique de $\mathbb{R}^n$, on en déduit que la i-ème colonne de A^{-1}, qui coïncide avec $A^{-1}b$, est à coefficients entiers. En variant $1 \leq i \leq n$, on obtient donc $A^{-1} \in \mathbb{Z}^{n\times n}$. En particulier, cela implique que $\det A^{-1} \in \mathbb{Z}$ et, comme $1 = \det A \det A^{-1}$, cela montre que $\det A$ divise 1, c'est-à-dire que $\det A = \pm 1$. □

Ce lemme motive la définition suivante qui est centrale dans cette section.

Définition 4.3.2 *On dit qu'une matrice carrée $A \in \mathbb{Z}^{n\times n}$ est* **unimodulaire** *quand $\det A = \pm 1$, et qu'une matrice générale $A \in \mathbb{Z}^{m\times n}$ est* **totalement unimodulaire** *quand toute sous-matrice carrée extraite de A est de déterminant ± 1 ou 0.*

En prenant des sous-matrices 1×1, on voit en particulier que les coefficients d'une matrice totalement unimodulaire valent nécessairement ± 1 ou 0. L'introduction des matrices totalement unimodulaires est motivée par le résultat suivant.

Proposition 4.3.3 *Soit le programme linéaire en nombres entiers (4.24). On suppose que les données $A \in \mathbb{Z}^{m\times n}$ et $b \in \mathbb{Z}^m$ sont entières et que la matrice A est totalement unimodulaire. Alors toute solution basique x de (4.24) est entière, c'est-à-dire que $x \in \mathbb{Z}^n$.*

Remarque 4.3.4 La Proposition 4.3.3 est très importante en pratique car elle implique que si l'on résout (4.24) par l'algorithme du simplexe "usuel", alors la solution que l'on obtiendra (ainsi que toutes les solutions intermédiaires) seront entières car le simplexe énumère des solutions basiques. Il n'y a rien à changer à l'algorithme du simplexe pour obtenir des solutions entières, du moins sous les hypothèses de la Proposition 4.3.3.

La Proposition 4.3.3 ne dit surtout pas que toutes les solutions optimales de (4.24) sont entières. D'ailleurs, il ne peut en être ainsi, à moins que la solution optimale ne soit unique, car, par linéarité du coût $c \cdot x$, tout barycentre de solutions optimales d'un programme linéaire est aussi solution optimale.

Il est intéressant de remarquer que l'on ne fait aucune hypothèse sur le vecteur c qui apparait dans le coût. Celui-ci peut ne pas être à valeurs entières mais, dans tous les cas, les solutions basiques (et donc une solution optimale au moins) seront entières. •

Démonstration. Rappelons qu'une solution basique correspond à un réarrangement des colonnes de A tel que $A = (B, N)$, où B est une matrice inversible $m \times m$ et N est une matrice $m \times (n-m)$, et que dans cette base on a $x = (x_B, x_N)$ avec $x_N = 0$, $x_B \geq 0$ et $Bx_B = b$. Comme A est totalement unimodulaire, B est unimodulaire et $x_B \in \mathbb{Z}^m$. Comme $x_N = 0$, on a bien $x \in \mathbb{Z}^n$. □

Bien sûr, pour que la Proposition 4.3.3 tienne toutes ses promesses, il faut vérifier qu'il existe "suffisamment" de matrices totalement unimodulaires. Il n'existe pas de caractérisation des matrices totalement unimodulaires mais on connait certaines conditions suffisantes. On donne ici une telle condition, très utile en pratique, due à Poincaré.

Lemme 4.3.5 *Si A est une matrice à coefficients ± 1 ou 0, avec au plus un coefficient 1 par colonne, et au plus un coefficient -1 par colonne, alors A est totalement unimodulaire.*

Démonstration. Comme l'hypothèse sur les valeurs des coefficients de A est aussi vraie pour toutes les sous-matrices de A, il suffit de considérer le cas où la matrice A est carrée et de vérifier que $\det A$ vaut ± 1 ou 0. Si A possède une colonne nulle, alors $\det A = 0$. Si A possède une colonne avec seulement un coefficient non-nul, on développe son déterminant par rapport à cette colonne, et l'on conclut par récurrence sur la dimension de A que $\det A \in \{\pm 1, 0\}$. Il reste donc à considérer le cas où chaque colonne de A a exactement un coefficient 1 et un coefficient -1 (tous les autres étant nuls). Ainsi, la somme des coefficients de chaque colonne est nulle. Autrement dit, le vecteur $(1, \cdots, 1)$ appartient au noyau de la transposée de A et donc $\det A = 0$. □

Nous allons voir dans les sous-sections suivantes que l'on rencontre souvent en pratique des matrices qui vérifient les hypothèses du Lemme 4.3.5 de Poincaré.

Exercice 4.3.1 Soit une matrice $A \in \mathbb{Z}^{m \times n}$ telle que les premiers éléments de chacune de ses colonnes sont des 1 alors que tous les autres éléments sont des 0. Autrement dit, pour tout $1 \leq j \leq n$, il existe $n_j \in \mathbb{N}$ tel que $a_{ij} = 1$ pour $1 \leq i \leq n_j$ et $a_{ij} = 0$

pour $n_j + 1 \leq i \leq m$. Montrer qu'une telle matrice (appelée matrice d'intervalles) est totalement unimodulaire. Indication : on vérifiera qu'il suffit de démontrer le résultat pour des matrices carrées et on fera une récurrence sur la taille de la matrice.

4.3.2 Introduction aux problèmes de flots

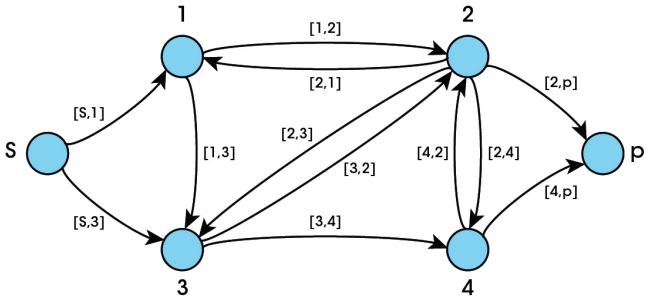


FIGURE 4.2 – Graphe orienté.

Nous faisons une brève présentation des problèmes de flots qui ne servent ici que de prétexte à illustrer la résolution de programmes linéaires en nombres entiers et, accessoirement, à donner des exemples de matrices vérifiant les hypothèses du Lemme 4.3.5 de Poincaré. Pour plus de détails sur les problèmes de flots et notamment pour d'autres algorithmes permettant de les résoudre, nous renvoyons à [5], [18]. Les problèmes de flots sont des problèmes d'optimisation sur des graphes, qui permettent de modéliser des réseaux (de transport, de communication, ou sociaux). Pour illustrer notre propos, imaginons qu'il s'agit d'un réseau de transport.

On introduit donc un graphe orienté $\mathcal{G} = (\mathcal{N}, \mathcal{A})$ où $\mathcal{N}$ est l'ensemble des **nœuds**, numérotés de 1 à m, et $\mathcal{A} \subset \mathcal{N} \times \mathcal{N}$ est l'ensemble des **arcs** ou arêtes qui relient deux nœuds distincts. Les nœuds sont par exemple des villes et les arcs des lignes de transport qui les relient. On note (i, j) l'arc qui relier le nœud i au nœud j. Le graphe est dit orienté car l'arc (i, j) n'est pas le même que l'arc (j, i) (voir la Figure 4.2) : cela permet de distinguer le sens du trajet, ce qui est utile dans de nombreuses applications. Tous les nœuds ne sont pas forcément reliés entre eux par un arc et on note n le nombre total d'arcs. Chaque arc $(i, j) \in \mathcal{A}$ possède une **capacité** maximale $u_{ij} \in \mathbb{R}_+ \cup \{+\infty\}$ (la quantité maximale de biens ou de personnes pouvant circuler sur cette ligne de transport) ainsi qu'un **coût** $c_{ij} \in \mathbb{R}$ (le prix unitaire du voyage sur cette ligne). Remarquez que, si la capacité maximale est bien positive, on laisse la possibilité d'un coût négatif en cas de subventions de certaines lignes... En chaque nœud i du graphe il est aussi possible d'avoir la donnée d'un flot extérieur $b_i \in \mathbb{R}$, c'est-à-dire une quantité de biens ou de personnes qui arrivent en i par d'autres moyens de transport que ceux modélisés par le graphe. Si $b_i > 0$, ce flot exogène est dit entrant, tandis que si $b_i < 0$, le flot exogène est sortant.

On appelle **flot** un vecteur $x \in \mathbb{R}^n$, défini sur sur $\mathcal{A}$ par $(i, j) \mapsto x_{ij}$, vérifiant

la **loi des nœuds de Kirchoff**

$$b_i + \sum_{j\in\mathcal{N},(j,i)\in\mathcal{A}} x_{ji} = \sum_{j\in\mathcal{N},(i,j)\in\mathcal{A}} x_{ij}, \qquad \forall i \in \mathcal{N}, \tag{4.25}$$

ainsi que la contrainte de positivité

$$0 \leq x_{ij}, \qquad \forall (i,j) \in \mathcal{A} \ . \tag{4.26}$$

La loi des nœuds de Kirchoff (4.25) exprime la conservation de la quantité de biens ou de personnes au nœud i : tout ce qui arrive en i (terme de gauche) égale ce qui en repart (terme de droite). En sommant en i les lois des nœuds (4.25), il apparait une condition nécessaire pour l'existence d'un flot, à savoir que la somme des flots extérieurs ou exogènes doit être nulle

$$\sum_{i\in\mathcal{N}} b_i = 0 \ . \tag{4.27}$$

Dans la suite, nous supposerons systématiquement que cette condition (4.27) est vérifiée. Un flot est dit **admissible** s'il satisfait en plus les contraintes de capacité

$$x_{ij} \leq u_{ij}, \qquad \forall (i,j) \in \mathcal{A} \ . \tag{4.28}$$

On appelle **problème de flot à coût minimum** le problème d'optimisation

$$\min_{x\in\mathbb{R}^n} \sum_{(i,j)\in\mathcal{A}} c_{ij}x_{ij} \quad \text{ sous les contraintes } \quad (4.25), (4.26), (4.28). \tag{4.29}$$

Il s'agit d'un programme linéaire qui admet toujours une solution si les coûts sont positifs, $c_{ij} \geq 0$, ou bien si les capacités sont finies, $u_{ij} < +\infty$. Pour donner une forme plus explicite de (4.29), sous la forme d'un programme linéaire, nous réécrivons la loi des nœuds de Kirchoff (4.25) comme $Ax = b$, où la matrice $A \in \mathbb{R}^{m\times n}$, appelée **matrice d'incidence nœuds-arcs** de $\mathcal{G}$, est définie par

$$A_{i,(j,k)} = \begin{cases} -1 & \text{si } i = k, \\ 1 & \text{si } i = j, \\ 0 & \text{sinon.} \end{cases} \tag{4.30}$$

La i-ème ligne de A est la loi de Kirchoff au nœud i, tandis que les élément non nuls de la (j,k)-ème colonne de A indiquent à quels nœuds (entrant ou sortant selon le signe) est relié l'arc (j,k). Le problème de flot (4.29) est donc équivalent à

$$\min_{x\in\mathbb{R}^n, Ax=b,\ 0\leq x\leq u} \sum_{(i,j)\in\mathcal{A}} c_{ij}x_{ij} \, . \tag{4.31}$$

Ce problème (4.31) n'est pas sous la forme standard (4.1) à cause de la contrainte de capacité maximale $x \leq u$. Néanmoins, grâce à l'introduction d'une variable d'écart

$y \in \mathbb{R}^n$, $y \geq 0$, la contrainte $x \leq u$ est équivalente à $x + y = u$. Par conséquent, (4.29) et (4.31) sont équivalents au programme linéaire sous forme standard

$$\min_{z \in \mathbb{R}^{2n}, \tilde{A}z = \tilde{b},\ 0 \leq z} \sum_{(i,j) \in \mathcal{A}} c_{ij} x_{ij}\,, \tag{4.32}$$

avec les notations $\tilde{b} \in \mathbb{R}^{m+n}$, $\tilde{A} \in \mathbb{R}^{(m+n)\times 2n}$ et

$$z = \begin{pmatrix} x \\ y \end{pmatrix}, \tilde{b} = \begin{pmatrix} b \\ u \end{pmatrix}, \tilde{A} = \begin{pmatrix} A & 0 \\ I_n & I_n \end{pmatrix}, \tag{4.33}$$

où I_n est la matrice identité dans $\mathbb{R}^{n\times n}$.

Remarque 4.3.6 Le problème de transport de l'Exemple 1.2.1 est en fait un exemple de problème de flot à coût minimum. Définissons le graphe dont les nœuds $\mathcal{N}$ sont les entrepôts, indicés par $1 \leq i \leq M$, et les clients, indicés par $1' \leq j' \leq N'$ (on rajoute un exposant $'$ pour distinguer les clients des entrepôts dans la liste des nœuds). Les arcs $\mathcal{A}$ de ce graphe orienté sont les arcs qui relient un entrepôt i à un client j', munis du coût c_{ij}. Il n'y a pas d'arcs entre clients, ni entre entrepôts, ni d'un client vers un entrepôt. Si les stocks sont égaux à la demande, $\sum_{i=1}^M s_i = \sum_{j=1}^N r_j$, alors les contraintes sur les quantités livrées $v_{ij} \geq 0$ sont la loi des nœuds de Kirchoff appliquée aux entrepôts i

$$\sum_{j=1}^{N} v_{ij} = s_i, \quad \text{pour } 1 \leq i \leq M,$$

et aux clients j

$$0 = \sum_{i=1}^{M} v_{ij} - r_j \quad \text{pour } 1 \leq j \leq N.$$

On choisit donc les flots externes comme $b_i = s_i$ pour les entrepôts et $b_{j'} = -r_{j'}$ pour les clients (notez le changement de signe de r_j pour prendre en compte le fait que les clients reçoivent alors que les entrepôts expédient). Avec le choix des capacités maximales infinies, $u_{ij} = +\infty$, le problème de transport de l'Exemple 1.2.1 est donc un problème de flot à coût minimum. Si les stocks sont supérieurs à la demande, $\sum_{i=1}^M s_i > \sum_{j=1}^N r_j$, alors il faut rajouter un client imaginaire qui reçoit tout le stock non consommé par les autres clients avec un coût de transport nul. Le problème est à nouveau un problème de flot à coût minimum. •

Il arrive souvent en pratique que l'on cherche des solutions entières d'un problème de flot. Par exemple, pour le problème de transport de l'Exemple 1.2.1, les marchandises à livrer peuvent être des colis, et livrer une fraction de colis peut ne pas avoir de sens. Il est donc naturel de se demander si un problème de flot à coût minimum, dont les données sont entières, a des solutions optimales qui sont aussi entières. Une réponse positive à cette question est apportée par le résultat suivant.

Théorème 4.3.7 *On suppose que les flots extérieurs ou exogènes sont entiers, $b_i \in \mathbb{Z}$, ainsi que les capacités, $u_{ij} \in \mathbb{N} \cup \{+\infty\}$. Alors les solutions basiques de* (4.32) *sont entières. En particulier, le problème* (4.29) *de flot à coût minimal admet une solution optimale entière $x \in \mathbb{N}^n$.*

Remarque 4.3.8 Une conséquence pratique importante du Théorème 4.3.7 est qu'on peut résoudre les problèmes de flots entiers par l'algorithme du simplexe. Non seulement la valeur optimale du coût est la même si on optimise sur les réels ou sur les entiers mais, en plus, comme l'algorithme du simplexe itère d'un sommet (ou solution basique) à un autre du polyèdre des solutions admissibles, les solutions trouvées par le simplexe sont entières. •

Démonstration. Si les capacités sont toutes infinies, $u_{ij} = +\infty$, alors il n'y a pas de contraintes $x \leq u$ et le programme linéaire (4.31) est déjà sous forme standard, sans nécessité d'introduire les variables d'écart y. Or le Lemme 4.3.9 ci-dessous montre que la matrice A est totalement unimodulaire. On conclut alors à l'aide de la Proposition 4.3.3.

Si certaines capacités sont finies, alors il faut raisonner sur le programme linéaire (4.32) où il n'est pas évident que la matrice $\tilde{A}$ est totalement unimodulaire. Néanmoins, si une solution basique z vérifie $\tilde{A}z = \tilde{b}$, alors, au vu de la structure (4.33) de $\tilde{A}$, on a $Ax = b$ et $y = u - x$. Comme A est totalement unimodulaire, x est entier, donc y et z aussi, ce qui conclut la preuve. □

Lemme 4.3.9 *La matrice d'incidence nœuds-arcs A d'un graphe, définie par* (4.30)*, est totalement unimodulaire.*

Démonstration. Par construction la matrice A a exactement un seul 1 et un seul -1 dans chacune de ses colonnes car chaque arc possède un unique nœud de départ et un unique nœud d'arrivée. On conclut grâce au Lemme 4.3.5 de Poincaré. □

Terminons cette section en donnant un exemple, très important en pratique, de problème de flot à coût minimum : le problème du **flot maximal**, qui s'intéresse seulement aux capacités et pas aux coûts. Dans le graphe $\mathcal{G}$ on distingue deux nœuds, notés s et p, appelés respectivement source et puits (voir la Figure 4.2), et on suppose que s n'a pas de prédécesseur, autrement dit $\{i \in \mathcal{N} \mid (i,s) \in \mathcal{A}\} = \emptyset$, et p n'a pas de successeur, c'est-à-dire $\{i \in \mathcal{N} \mid (p,i) \in \mathcal{A}\} = \emptyset$. Soit $v \in \mathbb{R}_+$. On appelle flot admissible de s à p de valeur v une solution x de $Ax = b$, $0 \leq x \leq u$ avec

$$b_i = \begin{cases} v & \text{si } i = s, \\ -v & \text{si } i = p, \\ 0 & \text{sinon.} \end{cases} \tag{4.34}$$

Le problème du flot maximal consiste à trouver un flot admissible de s à p de valeur v maximale. En d'autres termes, on cherche à faire passer de la source vers le puits le trafic maximal à travers le graphe $\mathcal{G}$.

Exercice 4.3.2 Montrer que le problème du flot maximal est bien un exemple de problème de flot à coût minimal. Pour cela, on rajoutera un arc reliant p à s au graphe

du problème du flot maximal, on annulera les deux flots externes b_s et b_p et on prendra les coûts $c_{ij} = 0$ sauf $c_{ps} = -1$.

Exercice 4.3.3 On reprend le graphe orienté $\mathcal{G}$ avec les deux nœuds, source s et puits p, tel que s n'a pas de prédécesseur et p n'a pas de successeur. On munit chaque arc (i, j) d'un coût $c_{ij} \geq 0$ qu'on interprète comme une distance entre les nœuds i et j. Le problème du **chemin de coût minimum** est de trouver une suite de nœuds consécutifs sur le graphe qui relient s à p et tels que la somme des coûts le long de ce chemin d'arcs est minimale. Montrer que ce problème peut se modéliser comme un problème de flot à coût minimum avec le choix (4.34) de flots externes pour $v = 1$ et des capacités maximales infinies.

4.3.3 Problème d'affectation

Rappelons le problème d'affectation, ou bien de mariage, tel que présenté dans l'Exemple 1.2.2. On a N femmes, indicées par $1 \leq i \leq N$, et N hommes, indicés par $1 \leq j \leq N$, avec une variable d'accord $a_{ij} \in \mathbb{R}^+$ qui indique l'attirance de i pour j. On cherche une permutation σ dans l'ensemble des permutations $\mathcal{S}_N$ de $\{1, ..., N\}$ qui réalise le maximum de

$$\max_{\sigma \in \mathcal{S}_N} \sum_{i=1}^{N} a_{i\sigma(i)}.$$

Bien sûr, comme le cardinal de l'ensemble $\mathcal{S}_N$ est $N!$, il est illusoire de vouloir résoudre ce problème d'affectation en énumérant simplement toutes les possibilités. C'est là une caractéristique des problèmes combinatoires où, même si l'espace de recherche est fini, sa grande taille rend impossible toute énumération exhaustive en pratique.

Si on introduit les variables de décision v_{ij}, qui vaut 1 s'il y a mariage entre la femme i et l'homme j et 0 sinon, on peut réécrire ce problème comme

$$\max_{(v_{ij})} \left(\sum_{i=1}^{N} \sum_{j=1}^{N} a_{ij} v_{ij} \right) \tag{4.35}$$

sous les contraintes

$$v_{ij} = 0 \text{ ou } 1, \ \sum_{j=1}^{N} v_{ij} = 1, \ \sum_{i=1}^{N} v_{ij} = 1 \ \text{ pour } 1 \leq i, j \leq N.$$

Les deux dernières contraintes égalité indiquent que chaque femme trouvera un mari et chaque homme une épouse (comme on maximise la réussite des mariages, on aurait pu écrire ces contraintes comme des inégalités aussi). Les matrices $v = (v_{ij})_{1 \leq i,j \leq N}$ qui vérifient ces contraintes sont précisément les matrices correspondantes aux permutations $\sigma \in \mathcal{S}_N$.

Si maintenant on relâche (ou relaxe) la contrainte $v_{ij} = 0$ ou 1, en $0 \leq v_{ij} \leq 1$, les matrices qui vérifient les contraintes relaxées

$$0 \leq v_{ij} \leq 1, \ \sum_{j=1}^{N} v_{ij} = 1, \ \sum_{i=1}^{N} v_{ij} = 1 \ \text{ pour } 1 \leq i, j \leq N \tag{4.36}$$

sont appelées matrices **doublement stochastiques**. Le problème (4.35) sous les contraintes (4.36) est un simple programme linéaire qu'on peut résoudre par l'algorithme du simplexe. Il est même posé sous forme standard car la contrainte $v_{ij} \leq 1$ est inutile au vu des contraintes égalité. Modulo le changement du max en min (il suffit de changer le signe des coûts a_{ij}), le problème (4.35) sous les contraintes (4.36) est un exemple de problème de transport qui, comme discuté à la Remarque 4.3.6, est un cas particulier de problème de flot à coût minimum. Pour montrer que ses solutions sont en fait entières, nous écrivons les contraintes égalité dans (4.36) sous la forme

$$\sum_{j=1}^{N} v_{ij} = 1 \text{ pour } 1 \leq i \leq N, \ 0 = \sum_{i=1}^{N} v_{ij} - 1 \ \text{ pour } 1 \leq j \leq N,$$

où on a effectué la même astuce que dans la Remarque 4.3.6 pour que la contrainte de mariage pour chaque homme j ressemble à la loi des nœuds de Kirchoff. On peut alors résumer ces contraintes d'égalité par $Av = b$ avec $b \in \mathbb{R}^{2N}$ le vecteur dont les N premières composantes sont égales à 1 et les N dernières égales à -1 et $A \in \mathbb{Z}^{2N \times N^2}$ est une matrice dont chacune des colonnes a exactement un élément égal à 1, un autre égal à -1 et tous les autres nuls. Le programme linéaire relaxé est donc

$$\max_{v \geq 0, \ Av = b} \left(\sum_{i=1}^{N} \sum_{j=1}^{N} a_{ij} v_{ij} \right). \tag{4.37}$$

En vertu du Lemme 4.3.5 de Poincaré, la matrice A est totalement unimodulaire. Par conséquent, la Proposition 4.3.3 garantit que les solutions optimales de (4.37) sont entières. Autrement dit, il suffit d'appliquer l'algorithme du simplexe au programme linéaire relaxé (4.37), sans aucune précaution particulière, pour résoudre le problème d'affectation qui est de nature combinatoire !

Remarque 4.3.10 Il existe une autre preuve du caractère entier des solutions optimales de (4.37) (voir [5]). L'ensemble des matrices doublement stochastiques, c'est-à-dire qui vérifient (4.36), est un ensemble convexe compact dont on peut montrer (théorème de Birkhoff - von Neumann) que les points extrêmes sont précisément les matrices de permutations. On peut aussi montrer (théorème de Minkowski) que l'ensemble des matrices doublement stochastiques est l'enveloppe convexe de ses points extrêmes. Par linéarité de la fonction coût, on en déduit qu'il existe des solutions qui sont des matrices de permutation, autrement dit des solutions optimales entières. •

Chapitre

ANNEXE : QUELQUES RAPPELS MATHÉMATIQUES

5.1 Rappels sur les espaces de Hilbert

Nous rappelons brièvement quelques propriétés des espaces de Hilbert (pour plus de détails, nous renvoyons au cours de mathématiques [17]). Pour simplifier la présentation, on ne considère que le cas d'espaces de Hilbert sur $\mathbb{R}$.

Définition 5.1.1 *Un espace de Hilbert réel est un espace vectoriel sur $\mathbb{R}$, muni d'un produit scalaire, noté $\langle x, y\rangle$, qui est complet pour la norme associée à ce produit scalaire, notée $\|x\| = \sqrt{\langle x, x\rangle}$. (On rappelle qu'un espace vectoriel normé est complet si toute suite de Cauchy est une suite convergente dont la limite appartient à cet espace.)*

Dans tout ce qui suit nous noterons V un espace de Hilbert réel, et $\langle x, y\rangle$ son produit scalaire associé.

Définition 5.1.2 *Un ensemble $K \subset V$ est dit convexe si, pour tout $x, y \in K$ et tout réel $\theta \in [0, 1]$, l'élément $(\theta x + (1-\theta)y)$ appartient à K.*

Un résultat essentiel est le théorème de projection sur un ensemble convexe (voir le théorème 10.1 de [17]).

Théorème 5.1.3 (de projection sur un convexe) *Soit V un espace de Hilbert. Soit $K \subset V$ un convexe fermé non vide. Pour tout $x \in V$, il existe un unique $x_K \in K$ tel que*

$$\|x - x_K\| = \min_{y \in K} \|x - y\|.$$

De façon équivalente, x_K est caractérisé par la propriété

$$x_K \in K, \langle x_K - x, x_K - y\rangle \leq 0 \quad \forall y \in K. \tag{5.1}$$

On appelle x_K la projection orthogonale sur K de x.

Remarque 5.1.4 Le Théorème 5.1.3 permet de définir une application P_K, appelée opérateur de projection sur l'ensemble convexe K, en posant $P_K x = x_K$. On vérifie sans peine que P_K est continue et faiblement contractante, c'est-à-dire que

$$\|P_K x - P_K y\| \leq \|x - y\| \quad \forall\, x, y \in V\ . \tag{5.2}$$

•

Remarque 5.1.5 Un cas particulier de convexe fermé K est un sous-espace vectoriel fermé W. Dans ce cas, la caractérisation (5.1) de x_W devient

$$x_W \in W, \langle x_W - x, z\rangle = 0 \quad \forall z \in W.$$

En effet, dans (5.1) il suffit de prendre $y = x_K \pm z$ avec z quelconque dans W. •

Démonstration. Soit y^n une suite minimisante, c'est-à-dire que $y^n \in K$ vérifie

$$d_n = \|x - y^n\| \to d = \inf_{y\in K} \|x - y\| \text{ quand } n \to +\infty.$$

Montrons que y^n est une suite de Cauchy. En utilisant la symétrie du produit scalaire, il vient

$$\|x - \frac{1}{2}(y^n + y^p)\|^2 + \|\frac{1}{2}(y^n - y^p)\|^2 = \frac{1}{2}(d_n^2 + d_p^2).$$

Or, par convexité de K, $(y^n + y^p)/2 \in K$, et $\|x - \frac{1}{2}(y^n + y^p)\|^2 \geq d^2$. Par conséquent

$$\|y^n - y^p\|^2 \leq 2(d_n^2 + d_p^2) - 4d^2,$$

ce qui montre que y^n est une suite de Cauchy. Comme V est un espace de Hilbert, il est complet, donc la suite y^n est convergente vers une limite x_K. Par ailleurs, comme K est fermé, cette limite x_K appartient à K. Par conséquent, on a $d = \|x - x_K\|$. Comme toute la suite minimisante est convergente, la limite est forcément unique, et x_K est le seul point de minimum de $\min_{y\in K} \|x - y\|$.

Soit $x_K \in K$ ce point de minimum. Pour tout $y \in K$ et $\theta \in [0, 1]$, par convexité de K, $x_K + \theta(y - x_K)$ appartient à K et on a

$$\|x - x_K\|^2 \leq \|x - (x_K + \theta(y - x_K))\|^2.$$

En développant le terme de droite, il vient

$$\|x - x_K\|^2 \leq \|x - x_K\|^2 + \theta^2\|y - x_K\|^2 - 2\theta\langle x - x_K, y - x_K\rangle,$$

ce qui donne pour $\theta > 0$

$$0 \geq -2\langle x - x_K, y - x_K\rangle + \theta\|z\|^2.$$

En faisant tendre θ vers 0, on obtient la caractérisation (5.1). Réciproquement, soit x_K qui vérifie cette caractérisation. Pour tout $y \in K$ on a

$$\|x - y\|^2 = \|x - x_K\|^2 + \|x_K - y\|^2 + 2\langle x - x_K, x_K - y\rangle \geq \|x - x_K\|^2,$$

ce qui prouve que x_K est bien la projection orthogonale de x sur K. □

Définition 5.1.6 *Soit V un espace de Hilbert pour le produit scalaire $\langle , \rangle$. On appelle base hilbertienne (dénombrable) de V une famille dénombrable $(e_n)_{n\geq 1}$ d'éléments de V qui est orthonormale pour le produit scalaire et telle que l'espace vectoriel engendré par cette famille est dense dans V.*

Remarque 5.1.7 L'existence d'une base hilbertienne dénombrable n'est pas garantie pour tous les espaces de Hilbert. Néanmoins, on peut construire des bases hilbertiennes pour les espaces de Hilbert séparables (i.e. qui contiennent une famille dénombrable dense). •

Proposition 5.1.8 *Soit V un espace de Hilbert pour le produit scalaire $\langle , \rangle$. Soit $(e_n)_{n\geq 1}$ une base hilbertienne de V. Pour tout élément x de V, il existe une unique suite $(x_n)_{n\geq 1}$ de réels telle que la somme partielle $\sum_{n=1}^{p} x_n e_n$ converge vers x quand p tend vers l'infini, et cette suite est définie par $x_n = \langle x, e_n\rangle$. De plus, on a*

$$\|x\|^2 = \langle x, x\rangle = \sum_{n\geq 1} |\langle x, e_n\rangle|^2. \tag{5.3}$$

On écrit alors

$$x = \sum_{n\geq 1} \langle x, e_n\rangle e_n.$$

Démonstration. S'il existe une suite $(x_n)_{n\geq 1}$ de réels telle que $\lim_{p\to+\infty} \sum_{n=1}^{p} x_n e_n = x$, alors par projection sur e_n (et comme cette suite est par définition indépendante de p) on a $x_n = \langle x, e_n\rangle$, ce qui prouve l'unicité de la suite $(x_n)_{n\geq 1}$. Montrons maintenant son existence. Par définition d'une base hilbertienne, pour tout $x \in V$ et pour tout $\epsilon > 0$, il existe y, combinaison linéaire finie des $(e_n)_{n\geq 1}$, tel que $\|x - y\| < \epsilon$. Grâce au Théorème 5.1.3 on peut définir une application linéaire S_p qui, à tout point $z \in V$, fait correspondre $S_p z = z_W$, où z_W est la projection orthogonale sur le sous-espace vectoriel W engendré par les p premiers vecteurs $(e_n)_{1\leq n\leq p}$. En vertu de (5.1), $(z - S_p z)$ est orthogonal à tout élément de W, donc en particulier à $S_p z$. On en déduit que

$$\|z\|^2 = \|z - S_p z\|^2 + \|S_p z\|^2, \tag{5.4}$$

ce qui implique

$$\|S_p z\| \leq \|z\| \forall z \in V.$$

Comme $S_p z$ est engendré par les $(e_n)_{1\leq n\leq p}$, et que $(z - S_p z)$ est orthogonal à chacun des $(e_n)_{1\leq n\leq p}$, on vérifie facilement que

$$S_p z = \sum_{n=1}^{p} \langle z, e_n\rangle e_n.$$

Pour p suffisamment grand, on a $S_p y = y$ car y est une combinaison linéaire finie des $(e_n)_{n\geq 1}$. Par conséquent

$$\|S_p x - x\| \leq \|S_p(x - y)\| + \|y - x\| \leq 2\|x - y\| \leq 2\epsilon.$$

On en déduit la convergence de $S_p x$ vers x. De cette convergence et de l'équation (5.4) on tire

$$\lim_{p\to+\infty} \|S_p x\|^2 = \|x\|^2,$$

qui n'est rien d'autre que la formule de sommation (5.3), dite de Parseval. □

Définition 5.1.9 *Soit V et W deux espaces de Hilbert réels. Une application linéaire A de V dans W est dite continue s'il existe une constante C telle que*

$$\|Ax\|_W \leq C\|x\|_V \quad \forall x \in V.$$

La plus petite constante C qui vérifie cette inégalité est la norme de l'application linéaire A, autrement dit

$$\|A\| = \sup_{x\in V, x\neq 0} \frac{\|Ax\|_W}{\|x\|_V}.$$

Souvent on utilisera la dénomination équivalente d'opérateur au lieu d'application entre espaces de Hilbert (on parlera ainsi d'opérateur linéaire continu plutôt que d'application linéaire continue). Si V est de dimension finie, alors toutes les applications linéaires de V dans W sont continues, mais ce n'est plus vrai si V est de dimension infinie.

Définition 5.1.10 *Soit V un espace de Hilbert réel. Son dual V' est l'ensemble des formes linéaires* **continues** *sur V, c'est-à-dire l'ensemble des applications linéaires continues de V dans $\mathbb{R}$. Par définition, la norme d'un élément $L \in V'$ est*

$$\|L\|_{V'} = \sup_{x\in V, x\neq 0} \frac{|L(x)|}{\|x\|}.$$

Dans un espace de Hilbert la dualité a une interprétation très simple grâce au théorème de Riesz (voir le théorème 10.3 de [17]) qui permet d'identifier un espace de Hilbert à son dual par isomorphisme.

Théorème 5.1.11 (de représentation de Riesz) *Soit V un espace de Hilbert réel, et soit V' son dual. Pour toute forme linéaire continue $L \in V'$ il existe un unique $y \in V$ tel que*

$$L(x) = \langle y, x\rangle \quad \forall x \in V.$$

De plus, on a $\|L\|_{V'} = \|y\|$.

Démonstration. Soit $M = \mathrm{Ker} L$. Il s'agit d'un sous-espace fermé de V car L est continue. Si $M = V$, alors L est identiquement nulle et seul $y = 0$ convient. Si $M \neq V$, alors il existe $z \in V \setminus M$. Soit alors $z_M \in M$ sa projection sur M. Comme z n'appartient pas à M, $z - z_M$ est non nul et, par le Théorème 5.1.3, est orthogonal à tout élément de M. Soit finalement

$$z_0 = \frac{z - z_M}{\|z - z_M\|}.$$

Tout vecteur $x \in V$ peut s'écrire

$$x = w + \lambda z_0 \text{ avec } \lambda = \frac{L(x)}{L(z_0)}.$$

On vérifie aisément que $L(w) = 0$, donc $w \in M$. Ceci prouve que $V = \text{Vect}(z_0) \oplus M$. Par définition de z_M et de z_0, on a $\langle w, z_0 \rangle = 0$, ce qui implique

$$L(x) = \langle x, z_0 \rangle L(z_0),$$

d'où le résultat désiré avec $y = L(z_0)z_0$ (l'unicité est évidente). D'autre part, on a

$$\|y\| = |L(z_0)|,$$

et

$$\|L\|_{V'} = \sup_{x \in V, x \neq 0} \frac{|L(x)|}{\|x\|} = L(z_0) \sup_{x \in V, x \neq 0} \frac{\langle x, z_0 \rangle}{\|x\|}.$$

Le maximum dans le dernier terme de cette égalité est atteint par $x = z_0$, ce qui implique que $\|L\|_{V'} = \|y\|$. □

Un résultat essentiel pour pouvoir démontrer le Lemme de Farkas 2.5.19 (utile en optimisation) est la propriété géométrique suivante qui est tout à fait conforme à l'intuition.

Théorème 5.1.12 (Séparation d'un point et d'un convexe) *Soit K une partie convexe non vide et fermée d'un espace de Hilbert V, et $x_0 \notin K$. Alors il existe un hyperplan fermé de V qui sépare strictement x_0 et K, c'est-à-dire qu'il existe une forme linéaire $L \in V'$ et $\alpha \in \mathbb{R}$ tels que*

$$L(x_0) < \alpha < L(x) \quad \forall\, x \in K\ . \tag{5.5}$$

Démonstration. Notons x_K la projection de x_0 sur K. Puisque $x_0 \notin K$, on a $x_K - x_0 \neq 0$. Soit L la forme linéaire définie pour tout $y \in V$ par $L(y) = \langle x_K - x_0, y \rangle$, et soit $\alpha = (L(x_k) + L(x_0))/2$. D'après (5.1), on a $L(x) \geq L(x_K) > \alpha > L(x_0)$ pour tout $x \in K$, ce qui achève la démonstration.

Bibliographie

[1] ALLAIRE G., *Analyse numérique et optimisation,* Éditions de l'École Polytechnique, Palaiseau (2005).

[2] AUBIN J.-P., *Mathematical methods of game and economic theory*, volume 7 of *Studies in Mathematics and its Applications*. North-Holland Publishing Co., Amsterdam-New York, 1979.

[3] BARDI M., CAPUZZO-DOLCETTA I., *Optimal control and viscosity solutions of Hamilton-Jacobi-Bellman equations*. Systems & Control : Foundations & Applications. Birkhäuser Boston, Inc., Boston, MA, 1997. With appendices by Maurizio Falcone and Pierpaolo Soravia.

[4] BONNANS J., *Optimisation continue,* Mathématiques appliquées pour le Master / SMAI, Dunod, Paris (2006).

[5] BONNANS J., GAUBERT S., *Recherche opérationnelle. Aspects mathématiques et applications,* Éditions de l'École Polytechnique, Palaiseau (2015).

[6] BONNANS J., GILBERT J.-C., LEMARECHAL C., SAGASTIZABAL C., *Optimisation numérique,* Mathématiques et Applications 27, Springer, Paris (1997).

[7] BONNANS J., ROUCHON P., *Commande et optimisation de systèmes dynamiques,* Éditions de l'École Polytechnique, Palaiseau (2005).

[8] BOSCAIN U., MIRRAHIMI M., *Automatic Control with Applications in Robotics and in Quantum Engineering,* cours de 3ème année à l'Ecole Polytechnique (2019).

[9] BREZIS H., *Functional analysis, Sobolev spaces and partial differential equations*. Universitext. Springer, New York, 2011.

[10] CARPENTIER P., COHEN G., *Décomposition-coordination en optimisation déterministe et stochastique,* Mathématiques et Applications 81, Springer, Paris (2017).

[11] CHAMBOLLE A., POCK T., *An introduction to continuous optimization for imaging,* Acta Numerica, pp. 161-319 (2016).

[12] CHVÁTAL V., *Linear programming*, Freeman and Co., New York (1983).

[13] CULIOLI J.-C., *Introduction à l'optimisation,* Éditions Ellipses, Paris (1994).

[14] ERN A., *Contrôle de modèles dynamiques,* cours de 2ème année à l'Ecole Polytechnique (2019).

[15] EKELAND I., TEMAM R., *Analyse convexe et problèmes variationnels,* Dunod, Paris (1974).

[16] FLETCHER R., *Practical methods of optimization. Vol. 1.* John Wiley & Sons, Ltd., Chichester, 1980. Unconstrained optimization, A Wiley-Interscience Publication.

[17] GOLSE F., LASZLO Y., PACARD F., VITERBO C., *Analyse réelle,* cours de 1ère année à l'Ecole Polytechnique (2019).

[18] GRIVA I., NASH S., SOFER A., *Linear and Nonlinear Optimization,* Second Edition, SIAM, Philadelphia (2009).

[19] HERMES H., LASALLE J. P., *Functional analysis and time optimal control.* Academic Press, New York-London, 1969. Mathematics in Science and Engineering, Vol. 56.

[20] ISIDORI A., *Nonlinear control systems.* Communications and Control Engineering Series. Springer-Verlag, Berlin, third edition, 1995.

[21] LEE E. B., MARKUS L., *Foundations of optimal control theory.* Robert E. Krieger Publishing Co., Inc., Melbourne, FL, second edition, 1986.

[22] LIONS P.-L., *Contrôle de modèles dynamiques.* Cours polycopié. École Polytechnique, 2016.

[23] NESTEROV Y., *Lectures on convex optimization,* Springer Optimization and Its Applications, 137, Springer, Cham, 2018.

[24] NOCEDAL J., WRIGHT S., *Numerical optimization,* Springer Series in Operations Research and Financial Engineering, New York (2006).

[25] PADBERG M., *Linear optimization and extensions*, Springer, Berlin (1999).

[26] ROCKAFELLAR R. T., WETS R. J.-B., *Variational analysis*, volume 317 of *Grundlehren der Mathematischen Wissenschaften [Fundamental Principles of Mathematical Sciences]*. Springer-Verlag, Berlin, 1998.

[27] SCHRIJVER A., *Theory of linear and integer programming*, Wiley, New York (1986).

[28] SONTAG E. D., *Mathematical control theory*, volume 6 of *Texts in Applied Mathematics*. Springer-Verlag, New York, second edition, 1998. Deterministic finite-dimensional systems.

[29] TRELAT E., *Contrôle optimal*. Mathématiques Concrètes. Vuibert, Paris, 2005. Théorie & applications.

[30] VINTER R., *Optimal control*. Systems & Control : Foundations & Applications. Birkhäuser Boston, Inc., Boston, MA, 2000.